

Understanding Race Conditions & asyncio.Lock

The Core Fundamentals

What is Asynchronous Programming?

- Asynchronous programming allows
 - a program to handle **multiple tasks concurrently**
 - **without waiting** for **each task** to **complete** before starting the next one.

The asyncio Model in Python

- Python's **asyncio** uses a **Single-Threaded** Event Loop based on **cooperative multitasking**.
- It does not execute code on multiple **CPU** cores at the exact same physical time.

How Task Switching Works (await)

- When an **async** function reaches an **await** expression e.g.
 - **waiting** for network I/O or
 - **asyncio.sleep()**
- It pauses its execution and **hands control back to** the Event Loop.
- The Event Loop then **picks** up **another** ready task to execute.

What is Shared State?

- Shared state refers to
 - any **variable, dictionary**
 - or database **record**
- That **multiple concurrent tasks** read from or write to during execution.

The Problem - Race Conditions

Definition of a Race Condition

- A **Race Condition** occurs when the **final result** of a program depends on the **uncoordinated timing** or sequence of execution among concurrent tasks accessing a shared resource
- **Shared Resource** ข้อมูลกลางที่ทุกคนเข้าถึงได้ (เช่น ตัวแปรคัลงคูปอง, ยอดเงิน ในบัญชี)
- **Critical Section** ช่วงที่มีการอ่าน/แก้ไข Shared Resource ซึ่งเป็น "โซนอันตราย" ที่ไม่ควรให้ใครเข้ามาแทรกกระทหว่างทำ

The Myth: "Single-Threaded Means Race-Condition-Free"

- Many developers **wrongly** assume single-threaded code cannot have race conditions.
- In **asyncio**, race conditions happen whenever
 - a task's execution is paused at an **await point**
 - while **mid-way through a read-modify-write** operation.

Anatomy of a Vulnerable Operation (Read-Modify-Write)

- A typical race condition happens in three steps:
 - **Read**: Task reads current state (e.g., inventory = 1).
 - **Pause** (await): Task pauses while waiting for external operation.
 - **Write**: Task updates state based on old reading (e.g., inventory -= 1).

Real-World Scenario: "The Last Coupon"

- Imagine 1 coupon remains in inventory (**count = 1**).
- **Student A** checks count (**$1 > 0 \rightarrow \text{Valid}$**).
- **Student A** pauses at an **await** point.
- **Student B** checks count before A finishes (**$1 > 0 \rightarrow \text{Valid}$**).
- **Both** students execute their **claim logic**, leading to **over-allocation** (**count = -1** or duplicate coupon assignment).

What is a Critical Section?

- A **Critical Section** is a specific region of **code** that **accesses** a **shared resource** and must not be executed by more than one task at a time to prevent data corruption.

The Solution - `asyncio.Lock`

What is asyncio.Lock?

- **asyncio.Lock** is a synchronization primitive used to enforce **Mutual Exclusion (Mutex)** in asynchronous code.
- The Lock Analogy (Restroom Key)
 - Think of a lock like a **single key to a restroom**
 - **Only** the person holding the key can enter.
 - If someone else wants to enter, they must **wait** in line until the key is returned.

Acquiring and Releasing Locks

- **Acquire**

- A task **requests** the lock. If available, it takes it and proceeds. If **occupied**, it pauses (await) until released.

- **Release**

- Once finished, the holding task releases the lock so the next waiting task can take it.

Using Asynchronous Context Managers (async with)

- In Python, locks are best managed using **async with lock** context manager syntax.
- This **guarantees** that the lock is **automatically released** even **if** an **exception** or **error** occurs inside the block.

```
# PROTECTED CRITICAL SECTION
async with coupon_lock:
    if company_coupons:
        await asyncio.sleep(0.01) # Safe pause, lock is held
        coupon = company_coupons.pop(0) # Guaranteed unique execution
```


Code Comparison & Impact

Without Lock (Unsafe Code)

```
# VULNERABLE TO RACE CONDITION
if company_coupons:
    await asyncio.sleep(0.01) # Context switch point!
    coupon = company_coupons.pop(0) # Double-pop can happen!
```

With Lock (Safe Code)

```
# PROTECTED CRITICAL SECTION
async with coupon_lock:
    if company_coupons:
        await asyncio.sleep(0.01) # Safe pause, lock is held
        coupon = company_coupons.pop(0) # Guaranteed unique execution
```

Key Differences: Without Lock vs. With Lock

- **Data Integrity**

- Without lock = Data corruption & crashes / With lock = 100% data consistency.

- **Execution Flow**

- Without lock = Interleaved execution inside critical section / With lock = Sequential access inside critical section.

- **Performance**

- Without lock = Faster execution (wrong results) / With lock = Minimal queuing delay (correct results).

Best Practices

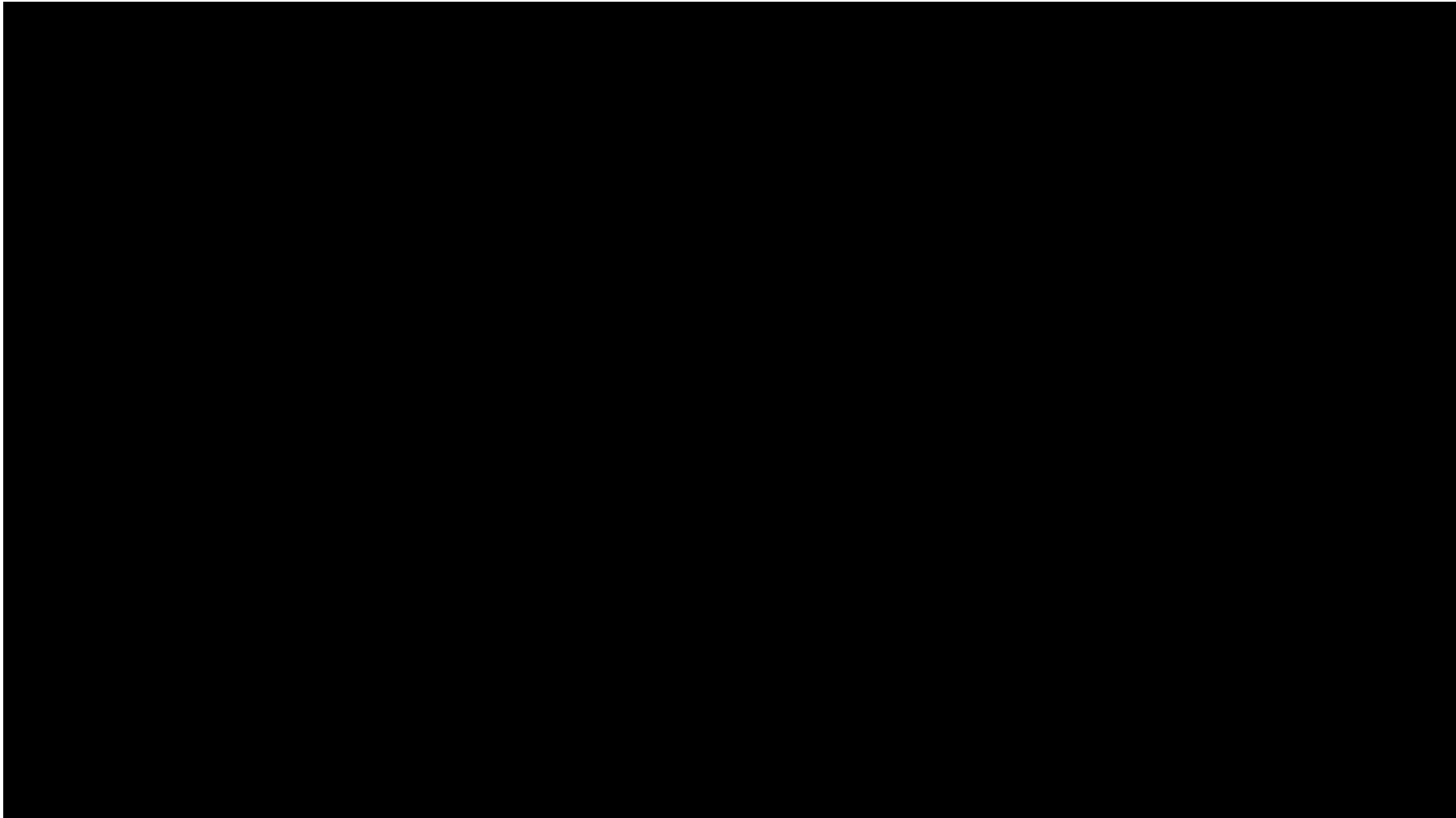
- **Keep Critical Sections Minimal**

- Only place essential read/write logic inside **async with lock:**
- **Avoid** locking heavy I/O tasks (like slow network calls) unless strictly necessary, as it turns your async code back into synchronous blocking code.

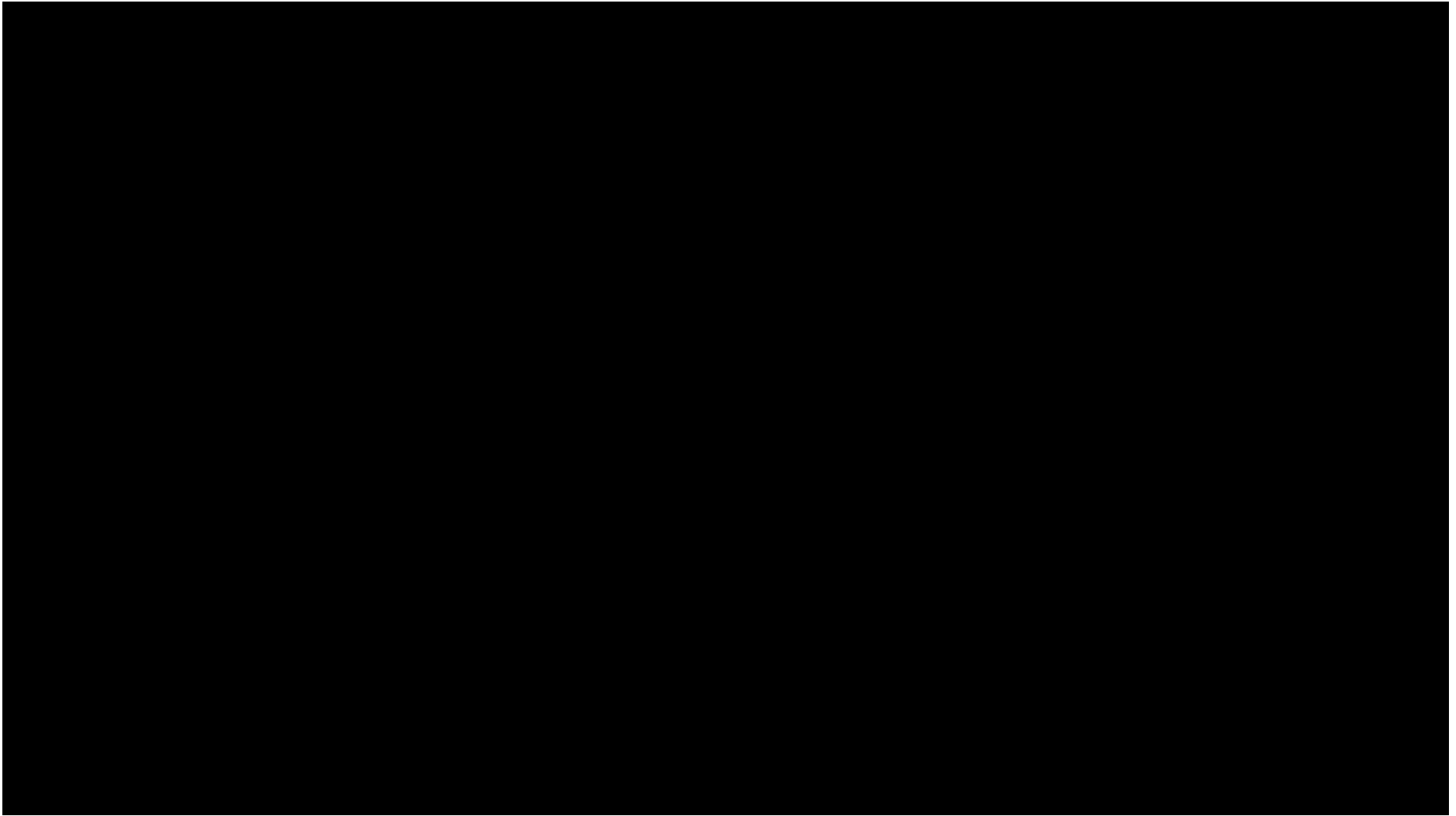
- **Summary Rule for Students**

- *"Identify your shared state, locate the await points inside operations on that state, and protect the read-modify-write cycle with an **asyncio.Lock**."*

Race Condition without Lock



Race Condition with asyncio.Lock



Assignment Team Coupon Hunting

Team Coupon Hunting

- ให้นักเรียนจับกลุ่ม กลุ่มละ 5-6 คน และตกลงแบ่งหน้าที่เพื่อจำลองระบบสถาปัตยกรรม Client-Server ดังนี้
 - **Server Machine 1** (ระบบที่มีช่องโหว่) นักเรียน 1 คน ในกลุ่ม ทำหน้าที่เปิดเครื่องตนเองเป็นแม่ข่ายรันไฟล์ `server_vulnerable.py` ซึ่งเป็นระบบที่ไม่ได้รับการปกป้องด้วย Lock
 - **Server Machine 2** (ระบบที่ปลอดภัย) นักเรียน 1 คน ในกลุ่ม ทำหน้าที่เปิดเครื่องตนเองเป็นแม่ข่ายรันไฟล์ `server.py` ซึ่งเป็นระบบที่ได้รับการแก้ไขปัญหาด้วย `asyncio.Lock`
 - **Client Hunters** (ลูกข่ายผู้แย่งชิง) นักเรียนทุกคนในกลุ่ม (รวมถึงผู้รับบท Server) จะต้องใช้เครื่องของตนเองรันไฟล์ `client.py` เพื่อยิง request แย่งชิงคูปอง (claim) ไปยัง Server เครื่องเพื่อนพร้อมกัน

กฎการกำหนดคลังคูปอง (Dynamic Inventory Constraint)

- เพื่อจำลองสถานะการแย่งชิงคูปองแบบต่อเนื่อง โดยอนุญาตให้นักเรียน 1 คนสามารถเก็บคูปองได้สูงสุดคนละ 2 ใบ ให้กำหนดจำนวนคูปองรวมใน Server ตามสูตร

$$\text{จำนวนคูปองรวมในคลัง} = (\text{จำนวนสมาชิกในกลุ่ม} \times 2) - 1$$

- ตัวอย่างผลลัพธ์ที่ถูกต้อง
 - กลุ่ม 5 คน: มีคูปอง 9 ใบ → นักเรียน 4 คนจะได้คนละ 2 ใบ และมี 1 คนได้เพียง 1 ใบ
 - กลุ่ม 6 คน: มีคูปอง 11 ใบ → นักเรียน 5 คนจะได้คนละ 2 ใบ และมี 1 คนได้เพียง 1 ใบ

การทดลองที่ 1: การสังเกตข้อผิดพลาดบน server_vulnerable.py

1. ให้ Server Machine 1 ตรวจสอบ IP Address ของเครื่องตนเองและประกาศให้เพื่อนในกลุ่มทราบ
2. ให้สมาชิกทุกคนปรับแต่ง SERVER_URL ในไฟล์ client.py ให้เรียกไปยัง IP ของ Server Machine 1
3. นัดเวลาทำการยิง Request POST /claim เข้าไปยัง Server พร้อมกันเพื่อเก็บคูปองสองใบ
4. สังเกตผลลัพธ์ที่เกิดขึ้น ทั้งใน Console ฝั่ง Server (เช่น การเกิด IndexError หรือ CRASH_BUG) และผลลัพธ์ฝั่ง Client (เช่น การได้รับคูปองเกินสิทธิ์ คูปองซ้ำรหัส หรือจำนวนคูปองในคลังติดลบ)

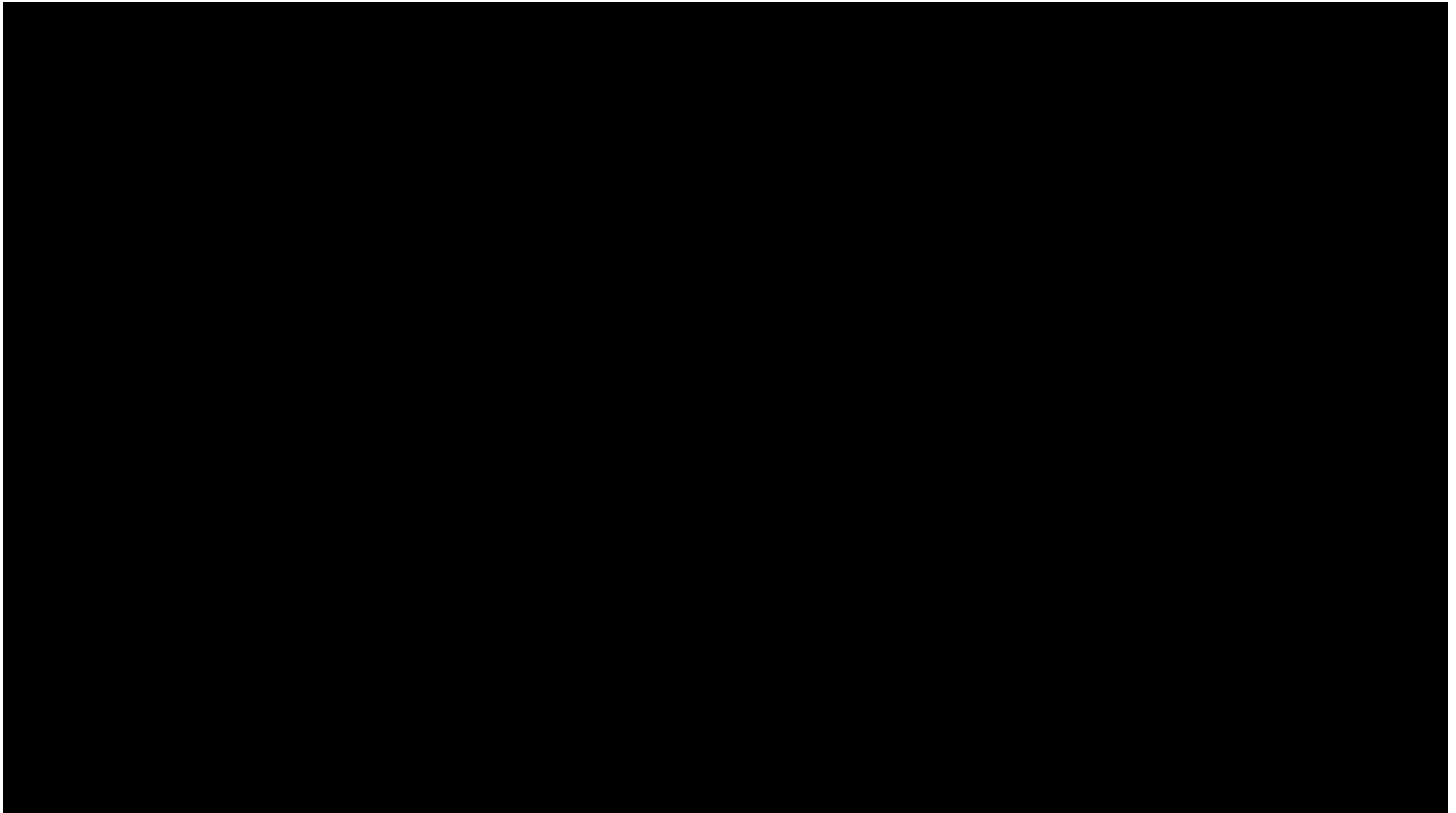
การทดลองที่ 2: การแก้ไขและยืนยันความถูกต้องบน server.py

1. เปลี่ยนไปเชื่อมต่อกับ Server Machine 2 ซึ่งมีการนำ `asyncio.Lock()` มาครอบครอบคลุมโซนการตรวจสอบและตัดจำนวนคุปอง (Critical Section)
2. ทำการยิง Request พร้อมกันอีกครั้งภายใต้เงื่อนไขเดิม (เก็บคุปองสูงสุดคนละ 2 ใบ)
3. บันทึกผลลัพธ์และยืนยันความถูกต้อง โดยระบบจะต้องแจกคุปองได้ครบตามจำนวน $(N \times 2) - 1$ ไม่มีการแจกคุปองซ้ำห้ส นักเรียนทุกคนต้องได้คุปองครบ 2 ใบ ยกเว้นซ้ำที่สุด 1 คนที่จะได้เพียง 1 ใบ และผู้ที่พยายามขอ ใบที่ 3 หรือผู้ที่มา claim ตอนคุปองหมดแล้วจะต้องได้รับสถานะปฏิเสธอย่างถูกต้อง 100%

เงื่อนไขการส่งงานรายบุคคล (Submission Deliverables)

- แม้นักเรียนจะทำการทดลองร่วมกันเป็นกลุ่ม แต่นักเรียนทุกคนจะต้องส่งไฟล์ซอร์สโค้ดฉบับสมบูรณ์เป็นรายบุคคล ประกอบด้วยไฟล์ดังต่อไปนี้
 1. **server_vulnerable.py** โค้ด Server ฝั่ง asynchronous ที่ไม่มีการใช้ Lock (แสดงจุดเสี่ยง Race Condition)
 2. **server.py** โค้ด Server ที่ได้รับการแก้ไขปัญหา Race Condition ด้วย asyncio.Lock เรียบร้อยแล้ว (ตั้งค่าคูปองแบบ $N \times 2 - 1$)
 3. **client.py** โค้ดฝั่ง Client ที่เขียนควบคุมการส่ง HTTP Request แบบ Concurrent ด้วย httpx.AsyncClient เพื่อพยายามเก็บคูปองให้ได้ 2 ใบ

server_lock_scene.py



server.py

